

UNIT — IV

Advanced MongoDB

Detailed Notes | End Semester Ready | 23CAF3010 NoSQL & MongoDB | SRM University Delhi-NCR

Topic	Coverage
1. Batch Processing	Bulk Write API — Ordered vs Unordered, bulkWrite() methods, performance
2. Data Aggregation	Pipeline stages, accumulators, expressions, MapReduce, single-purpose methods
3. Indexing	Types, createIndex, dropIndex, getIndexes, performance impact
4. Replication via Replica Sets	Architecture, Primary/Secondary/Oplog, Failover, Working, Election
5. Query by Embedded Field	Dot notation, nested document queries with all operators
6. Query by Field in Array	\$all, \$size, \$elemMatch, \$in on array fields
7. Specify Conditions with Operators	All comparison & logical operators with examples
8. Combine Conditions	\$and, \$or, \$nor, \$not, implicit AND, mixed combos

SECTION 1: Batch Processing

Definition

Batch Processing in MongoDB refers to performing operations on multiple documents at once rather than individually. It reduces round-trips to the server and dramatically improves throughput for large-scale data operations.

Batch Processing is essential when:

- You need to insert thousands of records from a data import.
- You need to update or delete many documents matching certain criteria.
- You want to minimize network latency by sending multiple operations in one call.
- You are processing ETL (Extract, Transform, Load) pipelines.

1.1 The Bulk Write API

MongoDB provides the Bulk Write API which allows multiple write operations (insert, update, delete, replace) to be performed in a single `db.collection.bulkWrite()` call. There are two modes:

Unordered Bulk Write	Ordered Bulk Write
<code>initializeUnorderedBulkOp()</code>	<code>initializeOrderedBulkOp()</code>
If one operation FAILS, MongoDB continues with the remaining operations	If one operation FAILS, MongoDB STOPS and does not execute remaining operations
Operations may be reordered by MongoDB for performance	Operations execute in EXACT order they were added
Faster — MongoDB can parallelize operations	Slower — strictly sequential
Use when: max throughput and order does not matter	Use when: strict order matters or stop-on-error is required
Returns BulkWriteResult with counts of successes/failures	Returns BulkWriteResult — stops at first failure

1.2 Unordered Batch Insert — Example

```
// Unordered bulk insert — continues even if one fails:
var bulk = db.students.initializeUnorderedBulkOp();
bulk.insert({ firstname: 'Sachin', lastname: 'Tendulkar' });
bulk.insert({ firstname: 'Virender', lastname: 'Sehwag' });
bulk.insert({ firstname: 'Rohit', lastname: 'Sharma' });
bulk.execute();

// BulkWriteResult: { nInserted: 3, nMatched: 0, ... }
```

1.3 Ordered Batch Insert — Example

```
// Ordered bulk insert — stops at first error:
var bulk = db.students.initializeOrderedBulkOp();
```

```
bulk.insert({ firstname: 'Robin', lastname: 'Manvin' });
bulk.insert({ firstname: 'John', lastname: 'Hudson' });
bulk.execute();
```

1.4 bulkWrite() — Modern API (All Operations Together)

The `bulkWrite()` method allows mixing `insertOne`, `updateOne`, `updateMany`, `replaceOne`, `deleteOne`, `deleteMany` in a single call:

```
db.pizzas.bulkWrite([
  { insertOne: { document: { _id:0, type:'Pepperoni', size:'Small', price:42 } } },
  { insertOne: { document: { _id:1, type:'Cheese', size:'Medium', price:73 } } },
  { updateOne: { filter: { type:'Cheese' }, update: { $set: { price: 80 } } } },
  { deleteOne: { filter: { type:'Pepperoni' } } },
  { replaceOne: { filter: { _id:1 }, replacement: { type:'Veggie', size:'Large', price:95 } } }
])

// All six bulk write operation types:
// insertOne, updateOne, updateMany, replaceOne, deleteOne, deleteMany
```

1.5 insertMany() — Simple Batch Insert

For pure inserts, `insertMany()` is the simplest batch approach without needing the Bulk API:

```
db.orders.insertMany([
  { item: 'Laptop', category: 'Electronics', price: 55000, qty: 1 },
  { item: 'Phone', category: 'Electronics', price: 20000, qty: 2 },
  { item: 'Notebook', category: 'Stationery', price: 50, qty: 10 }
])

// Ordered option (default true — stops on first error):
db.orders.insertMany([ ... ], { ordered: false }) // unordered
```

★ **Exam Tip:** *Batch Insert has TWO types: Ordered (stops on error) and Unordered (continues on error). `bulk.execute()` must always be called to actually run the operations. This is a direct MST question.*

SECTION 2: Data Aggregation

Aggregate Data	High-level data formed through the combination of numerical and non-numerical data from multiple sources. (From notes: 'Aggregate data is high level data formed through the combination of numerical and non-numerical data from multiple sources.')
Data Aggregation	The process of putting together a large group of data for high-level examination. (From notes: 'Data aggregation is the process of putting together a large group of data for high level examination.')
Aggregators	Organizations, websites or software applications that collect information from different sources and consolidate it in one place. (From notes exact definition)

Aggregation is a way of processing a large number of documents in a collection by means of passing them through different stages. The stages make up what is known as a pipeline. The stages in a pipeline can filter, sort, group, reshape and modify documents that pass through the pipeline.

One of the most common use cases of aggregation is to calculate aggregate values of groups of documents.

MongoDB Aggregation goes further through — it can also perform relational-like joins, reshape documents, create new & update existing collections, and so on.

2.1 MongoDB Aggregation Framework — Pipeline Diagram

From the handwritten notes, the aggregation pipeline flow is:



2.2 Types of Aggregation in MongoDB

Aggregation Type	Description
1. Aggregation Pipeline	The primary aggregation method. A series of stages (pipeline) where each stage transforms documents. Stages can filter, group, sort, reshape. Most powerful and flexible.
2. Single-Purpose Methods	Helper methods appended to find(). Quick to use but limited in scope. Examples: estimatedDocumentCount(), countDocuments(), distinct(). (From notes: 'These are called single-purpose methods like estimatedDocumentCount, count, distinct which are appended to find()')

3. MapReduce	Used for aggregating results for large volumes of data. Has two main functions: map (to group all the documents) and reduce (to perform operations on the grouped data). (From notes exact definition)
--------------	--

2.3 Aggregation Pipeline Stages — Complete List

From the handwritten notes — all stages with exact definitions:

Stage	From Notes (Exact)	Extended Description / Syntax
\$match	It is used to filter the documents. Can reduce the amount of documents that are given as input to the next stage.	{ \$match: { status: 'A', marks: { \$gt: 60 } } } — Place first for performance!
\$project	It is used to select some specific fields from a collection.	{ \$project: { name:1, marks:1, _id:0 } } — Include (1) or exclude (0) fields
\$group	It is used to group documents based on some value.	{ \$group: { _id: '\$course', total: { \$sum: 1 } } } — _id is the group-by field
\$sort	It is used to sort the document, that is rearranging them.	{ \$sort: { marks: -1 } } — 1 = ascending, -1 = descending
\$skip	It is used to skip n number of documents and passes the remaining documents.	{ \$skip: 10 } — skip first 10 documents
\$limit	It is used to limit/restrict the number of documents that are returned.	{ \$limit: 5 } — only pass 5 documents to next stage
\$unwind	It is used to unwind documents that are using an array. I.e. it deconstructs an array field in the document to return a document for each element.	{ \$unwind: '\$skills' } — one doc per array element
\$out	It is used to write results to a new collection.	{ \$out: 'resultCollection' } — saves pipeline output as a new collection

2.4 Expressions and Accumulators

From the handwritten notes — Expressions: It refers to the name of the field in input documents. For example: { \$id: 'total', \$sum: '\$fare' } — here \$id and \$fare are expressions.

Accumulators (used inside \$group stage):

Accumulator	From Notes (Exact)	Syntax Example
\$sum	It sums numeric values for the documents in each group.	totalMarks: { \$sum: '\$marks' } OR count: { \$sum: 1 }
\$count	It counts total number of documents.	Used as { \$sum: 1 } inside \$group, or { \$count: {} } stage
\$avg	It calculates the average of all given values from all documents.	avgMarks: { \$avg: '\$marks' }

\$min	It gets the minimum value from all the docs.	minMarks: { \$min: '\$marks' }
\$max	It gets the maximum value from all the docs.	maxMarks: { \$max: '\$marks' }
\$first	It gets the first document from the grouping.	firstStudent: { \$first: '\$name' }
\$last	It gets the last document from the grouping.	lastStudent: { \$last: '\$name' }

2.5 Aggregation Pipeline — Full Examples

Example 1: Count students per course

```
db.students.aggregate([
  { $group: { _id: '$course', studentCount: { $sum: 1 } } },
  { $sort: { studentCount: -1 } }
])
```

Example 2: Average marks per course (Active students only)

```
db.students.aggregate([
  { $match: { status: 'Active' } },      // filter first
  { $group: {
    _id: '$course',
    avgMarks: { $avg: '$marks' },
    maxMarks: { $max: '$marks' },
    minMarks: { $min: '$marks' },
    count: { $sum: 1 }
  } },
  { $sort: { avgMarks: -1 } }
])
```

Example 3: Total revenue per item (price × quantity)

```
db.orders.aggregate([
  { $group: {
    _id: '$item',
    totalRevenue: { $sum: { $multiply: ['$price', '$quantity'] } }
  } },
  { $sort: { totalRevenue: -1 } }
])
```

Example 4: Sunwind skills array then count per skill

```
db.students.aggregate([
  { $unwind: '$skills' },
  { $group: { _id: '$skills', count: { $sum: 1 } } },
  { $sort: { count: -1 } }
])
```

2.6 MapReduce

From the handwritten notes: MapReduce is used for aggregating results for the large volume of data.

MapReduce has two main functions:

- map — to group all the documents
- reduce — to perform operations on the grouped data

Syntax from notes: `db.CollectionName.mapreduce(mapping function, reduce function, $out: 'Result1234')`

```
// MapReduce syntax:
db.orders.mapReduce(
  function() { emit(this.item, this.price); }, // map: group by item
  function(key, values) { return Array.sum(values); }, // reduce: sum prices
  { out: 'order_totals' } // output collection
)
```

Note: *MapReduce is deprecated in newer MongoDB versions (v5.0+). The aggregation pipeline is preferred for all data processing tasks.*

2.7 MongoDB Aggregation Operations Process

From the handwritten notes: MongoDB aggregation operations process the data (records / documents) to return computed results. It collects values from various documents, groups them and then performs different types of operations on that grouped data like sum, average, min and max etc. to return a computed result.

Single-purpose aggregation operations are a collection of helper methods applied to a collection to calculate a result. These helper methods enable simple access to common aggregation processes.

Two of such methods provided are: `distinct()` and `countDocuments()`

```
// distinct() — returns unique values for a field:
db.students.distinct('course')
// Output: ['BTechCSE', 'BTechDS', 'BTechIT', 'BTechAI']

// countDocuments() — counts matching documents:
db.students.countDocuments({ status: 'Active' })

// estimatedDocumentCount() — fast approximate count:
db.students.estimatedDocumentCount()
```

★ Exam Tip: *Aggregation pipeline stages order: \$match FIRST (reduces documents early), then \$group, then \$sort, then \$project. \$match placed first = massive performance gain.*

SECTION 3: Indexing in MongoDB

From the handwritten notes: MongoDB uses indexing in order to make the query processing more efficient. If there is no indexing, then the MongoDB must scan every document in the collection to retrieve only those documents that match the query.

From the handwritten notes: Indexes are special data structures that stores some information related to the documents such that it becomes easy for MongoDB to find the right data file. The indexes are ordered by the value of the field specified in the index.

3.1 Why Indexing is Needed

- Without index: MongoDB performs a COLLSCAN — reads every document $O(n)$. Slow for large collections.
- With index: MongoDB performs an IXSCAN — traverses B-tree $O(\log n)$. Fast even for millions of docs.
- Indexes reduce CPU usage, disk I/O, and network overhead.
- Indexes are like the index at the back of a book — they let MongoDB jump to the right page directly.

3.2 Creating an Index — `createIndex()`

Syntax from handwritten notes: `db.collection_name.createIndex({ key: 1 })`

```
// Single field ascending index (1 = ascending, -1 = descending):
db.students.createIndex({ name: 1 })

// Single field descending:
db.orders.createIndex({ price: -1 })

// Compound index (multiple fields):
db.orders.createIndex({ item: 1, price: -1 })

// Unique index:
db.students.createIndex({ email: 1 }, { unique: true })

// TTL index — auto-delete after 30 days:
db.logs.createIndex({ createdAt: 1 }, { expireAfterSeconds: 2592000 })
```

3.3 Dropping an Index — `dropIndex()`

Syntax from handwritten notes: `db.NameCollection.dropIndex({ key: 0 })`

```
// Drop a specific index:
db.students.dropIndex({ name: 1 })

// Drop all indexes (except _id):
db.students.dropIndexes()
```

3.4 Get Description of All Indexes — `getIndexes()`

From the handwritten notes: The `getIndexes()` method in MongoDB gives a description of all the indexes that exist in the given collection.

Syntax from notes: `db.Name.collection.getIndexes()`

```
// List all indexes on a collection:
db.students.getIndexes()

// Sample output:
// [
//   { key: { _id: 1 }, name: '_id_ ' },
//   { key: { name: 1 }, name: 'name_1' }
// ]
```

3.5 Index Types

From the handwritten notes — 5 index types covered:

1. Single Field Index

From notes: Single field indexes collect and sort data from a single field in each document in a collection.

```
db.students.createIndex({ marks: 1 }) // ascending index on marks
db.students.createIndex({ name: -1 }) // descending index on name
```

2. Compound Index

From notes: Compound indexes collect and sort data from two or more fields, in each document in a collection. Data is grouped by the first field in the index and then by each subsequent field.

```
// Index on course ascending, then marks descending:
db.students.createIndex({ course: 1, marks: -1 })

// MongoDB uses the leftmost prefix rule:
// This index supports queries on: {course}, {course, marks}
// But NOT on: {marks} alone
```

3. Multikey Index

From notes: Multikey indexes collect and sort data stored in arrays.

```
// Automatically created when you index an array field:
db.students.createIndex({ skills: 1 })
// Now each element of the skills array is indexed separately.
// Supports queries like: db.students.find({ skills: 'Python' })
```

4. Geospatial Index

From notes: Geospatial indexes improve performance for queries on geospatial coordinate data.

- 2D Indexes — uses planar geometry to return results (flat 2D maps).
- 2DSphere Indexes — uses spherical geometry to return results (Earth-like globe).

```
// 2D index for flat coordinate queries:
db.places.createIndex({ location: '2d' })

// 2DSphere index for geographic queries:
```

```
db.places.createIndex({ location: '2dsphere' })
```

5. Text Index

From notes: Text indexes support text search queries on fields containing string content.

```
// Text index for full-text search:
db.articles.createIndex({ content: 'text' })

// Search using text index:
db.articles.find({ $text: { $search: 'MongoDB aggregation' } })
```

3.6 Index Types Summary Table

Index Type	Description	Use Case
Single Field	Index on one field — most common	Query by name, marks, status
Compound	Index on 2+ fields — grouped by first field then subsequent	Multi-field queries: course + marks
Multikey	Auto-created for array fields — indexes each element	Query: skills contains 'Python'
Geospatial 2D	Planar geometry for 2D coordinate data	Flat map location queries
Geospatial 2DSphere	Spherical geometry for globe-like coordinates	GPS/location-based apps
Text	Full-text search on string fields	Search article content, descriptions
TTL	Auto-deletes documents after specified time	Session data, logs, caches
Unique	Prevents duplicate values in a field	Email, username uniqueness

★ **Exam Tip:** *getIndexes()* = description of all indexes. *dropIndex()* = drop a specific one. *createIndex({key:1})* = create ascending. *createIndex({key:-1})* = descending. These exact syntaxes appear in MST questions.

SECTION 4: Replication via Replica Sets

4.1 What is Replication?

From the handwritten notes: Replication refers to the practice of maintaining multiple copies of data across different nodes or servers to ensure high availability & fault-tolerance and load-balancing. Replication in NoSQL databases is essential for ensuring data durability & system resilience.

From the handwritten notes: Replication is the method of duplication of data across multiple servers.

Example from notes: We have an application that reads and writes data to the database and says Server A has a name & balance which will be copied / replicated to 2 other servers in 2 different locations.

From notes: Replication increases redundancy and data availability with multiple copies of data on different database servers. So it will increase the performance of reading scaling.

4.2 Features of Replication

From the handwritten notes — Features of Replication:

1. Replica sets are the clusters of N different nodes that maintain the same copy of the data set.
2. The Primary Server receives all write operations and records all changes of the data i.e. oplog.
3. The secondary members then copy and apply these changes in an asynchronous process.
4. All the secondary nodes are connected with the primary nodes. There is one heartbeat signal from the primary nodes. If the primary server goes down, an eligible secondary will hold the new primary.

4.3 Replica Set Architecture

From the handwritten notes: The set of servers that maintain the same copy of data is known as replica servers.

Replica Server (Server C)	Server A — Primary Server	Replica Server (Server B)
Name: Balance (replica copy)	Name: Balance (original) Receives ALL Reads & Writes	Name: Balance (replica copy)

***Note:** Application reads from Primary (default) or Secondaries (with readPreference). All writes go to Primary only.*

4.4 Replica Set Components — Primary, Secondary, Oplog

Primary Node

From the handwritten notes: The Primary node is responsible for receiving all write operations (inserts, updates, deletes) from clients. It also replicates data to the secondary nodes to ensure data consistency.

- Receives ALL write operations from clients.
- Only ONE primary in a replica set at any time.

- Maintains the oplog (operations log) — records every write in chronological order.
- Clients read from primary by default (strong consistency).

Secondary Node

From the handwritten notes: Secondary nodes replicate data from the primary node using an asynchronous replication process. They can also serve read operations, distributing the load & improving read scalability.

- Replicates data from primary via the oplog asynchronously.
- Can serve read requests (with appropriate readPreference setting).
- Distributes read load across the replica set — improves read scalability.
- Eligible to become primary if current primary fails.

Oplog (Operations Log)

From the handwritten notes: The Primary node also maintains an oplog, which is a capped collection that records all write operations in a chronological order. Secondary nodes use the oplog to catch up with the primary node & apply the same operation to maintain data consistency.

- oplog is a special capped collection on the primary (fixed size — oldest entries overwritten when full).
- Records every insert, update, delete in order.
- Secondaries continuously poll the oplog and apply operations to their copy of the data.
- If a secondary falls behind, it catches up by reading missed oplog entries.
- If oplog overflows (secondary offline too long), a full initial sync is required.

4.5 Failover Mechanism

From the handwritten notes: In the event of primary node failure, a secondary node can be elected as the new primary node through an automated failover process. This ensures continuous availability & seamless operations of the database.

From the handwritten notes — How election works:

5. The Primary node is unavailable or inoperable.
6. A secondary node will take the primary nodes role to provide continuous availability of data.
7. In such cases, the primary node selection is made through a process called Replica Set Elections where the most suitable secondary node is selected as the new primary node.
8. While the Primary node is the only instance that accepts write operations, any other node within a replica set can accept read operations.

4.6 Working of Replica Sets

From the handwritten notes — Working: MongoDB handles replication through a replica set, which consists of multiple MongoDB nodes that are grouped together as a unit.

9. One of the nodes will be considered the Primary node that receives all the write operations.
10. The others are considered secondary nodes. These secondary nodes will replicate the data from the primary node.

11. While the Primary node is the only instance that accepts write operations, any other node within a replica set can accept read operations.
12. In an event where the Primary node is unavailable or inoperable, a secondary node will take the primary node role to provide continuous availability of data.
13. In such cases, the primary node selection is made through a process called Replica Set Elections where the most suitable secondary node is selected as the new primary node.

4.7 Replica Set Requirements and Commands

From the handwritten notes: A replica set requires a minimum of three MongoDB nodes.

Replica Set Command	Purpose
rs.initiate()	Initialize a new replica set
rs.status()	Check current status of all replica set members
rs.add('hostname:port')	Add a new secondary member to the set
rs.addArb('hostname:port')	Add an arbiter (votes only, no data)
rs.conf()	View current replica set configuration
rs.stepDown()	Force current primary to step down (trigger election)
rs.isMaster()	Returns info about primary/secondary status

```
// Initialize replica set:
rs.initiate()

// Check who is primary:
rs.status()

// Add secondary:
rs.add('mongodb1.example.com:27017')

// Add arbiter (breaks voting ties, no data stored):
rs.addArb('arbiter.example.com:27017')
```

★ **Exam Tip:** A replica set requires *MINIMUM 3 nodes*. Primary fails = Secondary elected automatically. Oplog is a *CAPPED* collection on the primary. Secondaries replicate *ASYNCHRONOUSLY*. All writes go to *PRIMARY ONLY*.

SECTION 5: Query by a Field in an Embedded Document

An embedded document (also called nested document or sub-document) is a document stored as the value of a field inside another document. To query fields inside embedded documents, MongoDB uses dot notation.

Dot Notation

The method used to access fields inside embedded/nested documents. Format: 'parentField.nestedField' — must be enclosed in quotes when used as a query key.

5.1 Sample Collection with Embedded Documents

```
// employees collection with embedded address:
db.employees.insertMany([
  { name:'Tamanna', salary:50000, address:{ city:'Delhi', pin:'110001' } },
  { name:'Akash', salary:60000, address:{ city:'Mumbai', pin:'400001' } },
  { name:'Priya', salary:55000, address:{ city:'Delhi', pin:'110002' } },
  { name:'Karan', salary:45000, address:{ city:'Pune', pin:'411001' } }
])
```

5.2 Query by Exact Match on Embedded Field

```
// Find all employees where address.city = 'Delhi':
db.employees.find({ 'address.city': 'Delhi' })

// SQL equivalent: SELECT * FROM employees WHERE address_city = 'Delhi'

// Find by exact embedded document match (order matters!):
db.employees.find({ address: { city:'Delhi', pin:'110001' } })
// NOTE: This matches ONLY if document has exactly these fields in this order.
```

5.3 Query Embedded Field with Comparison Operators

```
// Find employees in Delhi with salary > 50000:
db.employees.find({
  'address.city': 'Delhi',
  salary: { $gt: 50000 }
})

// Find employees NOT in Delhi:
db.employees.find({ 'address.city': { $ne: 'Delhi' } })

// Find employees in Delhi OR Mumbai:
db.employees.find({ 'address.city': { $in: ['Delhi', 'Mumbai'] } })
```

5.4 Update Embedded Field Using Dot Notation

```
// Update city to 'Bangalore' for employee Akash:
db.employees.updateOne(
  { name: 'Akash' },
  { $set: { 'address.city': 'Bangalore', 'address.pin': '560001' } }
```

```
)

// Update ALL employees in Delhi to Mumbai:
db.employees.updateMany(
  { 'address.city': 'Delhi' },
  { $set: { 'address.city': 'Mumbai' } }
)
```

5.5 Multi-Level Nesting

```
// Deeply nested document:
db.company.insertOne({
  name: 'TechCorp',
  location: { country: 'India', state: { name: 'Delhi', code: 'DL' } }
})

// Query deeply nested field:
db.company.find({ 'location.state.name': 'Delhi' })
```

★ **Exam Tip:** *ALWAYS use quotes around dot-notation keys: { 'address.city': 'Delhi' }. Without quotes it is a syntax error. Dot notation works in find(), update (\$set), sort, project — everywhere.*

SECTION 6: Query by a Field in an Array

MongoDB stores array fields natively. Querying array fields is powerful — you can find documents where an array contains a specific value, matches multiple values, has a specific size, or where array elements match complex conditions.

6.1 Sample Collection with Array Fields

```
db.students.insertMany([
  { name:'Akash', marks:85, skills:['Python','MongoDB','C++'], tags:['topper','active'] },
  { name:'Priya', marks:92, skills:['Python','ML','TensorFlow'], tags:['topper','active'] },
  { name:'Rahul', marks:62, skills:['Java','React'], tags:['active'] },
  { name:'Neha', marks:88, skills:['Python','MongoDB','ML'], tags:['topper','active'] }
])
```

6.2 Basic Array Query — Element Match

```
// Find students who have 'Python' in skills array:
db.students.find({ skills: 'Python' })
// MongoDB automatically checks if 'Python' is an ELEMENT of the skills array.

// Find students who have 'topper' tag:
db.students.find({ tags: 'topper' })
```

6.3 \$all — Array Contains ALL Specified Values

```
// Find students who know BOTH Python AND MongoDB:
db.students.find({ skills: { $all: ['Python', 'MongoDB'] } })

// Find students who are BOTH 'topper' AND 'active':
db.students.find({ tags: { $all: ['topper', 'active'] } })
```

6.4 \$size — Array Has Exact Number of Elements

```
// Find students whose skills array has exactly 3 elements:
db.students.find({ skills: { $size: 3 } })

// Find students with exactly 2 tags:
db.students.find({ tags: { $size: 2 } })
```

6.5 \$elemMatch — Array Element Satisfies Multiple Conditions

```
// Sample: scores array with sub-documents
db.results.insertOne({
  student: 'Akash',
  scores: [ { subject:'Math', marks:85 }, { subject:'Science', marks:90 } ]
})

// Find students with at least one score where marks > 80 AND subject = 'Math':
```

```
db.results.find({
  scores: { $elemMatch: { subject: 'Math', marks: { $gt: 80 } } }
})

// WITHOUT $elemMatch (incorrect for multi-condition per element):
// { scores.subject:'Math', scores.marks:{ $gt:80 } } — may match different elements!
```

6.6 \$in on Array Fields

```
// Find students who know AT LEAST ONE of: Python, Java, ML:
db.students.find({ skills: { $in: ['Python', 'Java', 'ML'] } })
```

```
// Find students who do NOT know Java or C++:
db.students.find({ skills: { $nin: ['Java', 'C++'] } })
```

6.7 Array Query with Projection

```
// Return only name and first skill (index 0):
db.students.find({}, { name:1, 'skills.0':1, _id:0 })

// $slice — return only first 2 elements of skills array:
db.students.find({}, { name:1, skills: { $slice: 2 } })

// $slice — return last 1 element:
db.students.find({}, { skills: { $slice: -1 } })
```

★ **Exam Tip:** *\$all* = array must contain ALL values listed. *\$elemMatch* = single array element must satisfy ALL conditions. *\$size* = exact element count. *\$in* = array contains AT LEAST ONE of listed values.

SECTION 7: Specify Conditions with Operators

MongoDB provides a rich set of query operators to specify conditions. These operators are used as values in the filter document passed to `find()`, `updateOne()`, `deleteMany()`, and aggregation `$match` stages.

7.1 Comparison Operators

Operator	Meaning	Example
<code>\$eq</code>	Equal to (default when no operator used)	<code>{ age: { \$eq: 20 } }</code> same as <code>{ age: 20 }</code>
<code>\$ne</code>	Not equal to	<code>{ status: { \$ne: 'Inactive' } }</code>
<code>\$gt</code>	Greater than	<code>{ marks: { \$gt: 80 } }</code>
<code>\$gte</code>	Greater than or equal to	<code>{ marks: { \$gte: 80 } }</code>
<code>\$lt</code>	Less than	<code>{ age: { \$lt: 25 } }</code>
<code>\$lte</code>	Less than or equal to	<code>{ price: { \$lte: 500 } }</code>
<code>\$in</code>	Value is in specified array	<code>{ status: { \$in: ['A','P'] } }</code>
<code>\$nin</code>	Value is NOT in specified array	<code>{ skills: { \$nin: ['COBOL'] } }</code>

```
// Marks between 60 and 90 (range — implicit AND on same field):
db.students.find( { marks: { $gt: 60, $lt: 90 } } )
```

```
// Salary >= 50000:
db.employees.find( { salary: { $gte: 50000 } } )
```

```
// Status not 'Inactive':
db.students.find( { status: { $ne: 'Inactive' } } )
```

```
// course is BTechCSE or BTechDS:
db.students.find( { course: { $in: ['BTechCSE', 'BTechDS'] } } )
```

7.2 Logical Operators

Operator	Meaning	Syntax
<code>\$and</code>	ALL conditions must be true (explicit)	<code>{ \$and: [{cond1}, {cond2}] }</code>
<code>\$or</code>	AT LEAST ONE condition must be true	<code>{ \$or: [{cond1}, {cond2}] }</code>
<code>\$not</code>	Negates a condition	<code>{ field: { \$not: { \$gt: 100 } } }</code>
<code>\$nor</code>	NONE of the conditions must be true	<code>{ \$nor: [{cond1}, {cond2}] }</code>

```
// $and — explicit:
db.students.find( { $and: [ { course:'BTechCSE' }, { marks:{ $gt:75 } } ] } )
```

```
// Implicit AND (shorthand — same result):
db.students.find( { course: 'BTechCSE', marks: { $gt: 75 } } )
```

```
// $or:
db.students.find( { $or: [ { marks:{ $gt:90 } }, { skills:'Python' } ] } )
```

```
// $nor — neither condition true:
db.students.find({ $nor: [ { status:'Inactive' }, { marks:{ $lt:40 } } ] })

// $not:
db.students.find({ marks: { $not: { $gt: 80 } } }) // marks <= 80
```

7.3 Element Operators

Operator	Meaning & Example
\$exists	Check if field exists: { phone: { \$exists: true } }
\$type	Check field data type: { age: { \$type: 'int' } } or { age: { \$type: 16 } }

```
// Find students who HAVE a phone field:
db.students.find({ phone: { $exists: true } })

// Find students where age is stored as integer type (BSON type 16):
db.students.find({ age: { $type: 'int' } })

// Find documents where marks is a double:
db.students.find({ marks: { $type: 'double' } })
```

7.4 Evaluation Operators

Operator	Meaning & Example
\$regex	Pattern match on strings: { name: { \$regex: /^Ak/i } }
\$where	JavaScript expression: { \$where: 'this.marks > 80' } (deprecated)
\$expr	Use aggregation expressions in query: { \$expr: { \$gt: ['\$marks', '\$passMark'] } }

```
// Names starting with 'A' (case insensitive):
db.students.find({ name: { $regex: '^A', $options: 'i' } })

// $expr — compare two fields:
db.orders.find({ $expr: { $gt: ['$revenue', '$cost'] } })
```

★ **Exam Tip:** *Implicit AND is shorthand — { a:1, b:2 } = { \$and:[{a:1},{b:2}] }. \$in checks if a value is in an array. \$nin is the opposite. \$exists:true finds docs where field exists. \$type checks BSON type.*

SECTION 8: Combine Conditions

MongoDB allows combining multiple query conditions using logical operators (\$and, \$or, \$nor, \$not) and implicit AND. Complex queries combine comparisons, logical operators, array conditions, and embedded field queries.

8.1 Implicit AND — Multiple Conditions on Different Fields

When you list multiple fields in a filter document, MongoDB treats them as AND — all conditions must be true. This is the most common and concise way to combine conditions.

```
// All three conditions must be true:
db.students.find({
  course: 'BTechCSE',    // course = BTechCSE
  marks: { $gt: 75 },    // AND marks > 75
  status: 'Active'      // AND status = Active
})
```

8.2 Explicit \$and — Multiple Conditions Including Same Field

Use explicit \$and when you need multiple conditions on the same field, or when combining \$or with other conditions.

```
// Explicit $and — same as implicit for different fields:
db.students.find({
  $and: [
    { course: 'BTechCSE' },
    { marks: { $gt: 75 } },
    { status: 'Active' }
  ]
})

// Explicit $and required when combining $or + other conditions:
db.students.find({
  $and: [
    { 'address.city': 'Delhi' },
    { $or: [
      { marks: { $gt: 85 } },
      { skills: 'MongoDB' }
    ] }
  ]
})
```

8.3 \$or — At Least One Condition Must Match

```
// Students with marks > 90 OR who know Python:
db.students.find({
  $or: [
    { marks: { $gt: 90 } },
    { skills: 'Python' }
  ]
})

// Students in Delhi OR Mumbai:
db.students.find({
```

```

$or: [
  { 'address.city': 'Delhi' },
  { 'address.city': 'Mumbai' }
]
})
// Simpler shorthand with $in:
db.students.find({ 'address.city': { $in: ['Delhi','Mumbai'] } })

```

8.4 Complex Combined Conditions — Full Examples

Example 1: BTechCSE OR BTechDS students with marks 60–90 who know Python

```

db.students.find({
  $and: [
    { course: { $in: ['BTechCSE','BTechDS'] } },
    { marks: { $gt: 60, $lt: 90 } },
    { skills: 'Python' }
  ]
})

```

Example 2: Active students in Delhi with marks > 75 OR students with MongoDB skill

```

db.students.find({
  $or: [
    {
      $and: [
        { status: 'Active' },
        { 'address.city': 'Delhi' },
        { marks: { $gt: 75 } }
      ]
    },
    { skills: 'MongoDB' }
  ]
})

```

Example 3: Electronics orders with price > 10000, quantity > 1, and status not Cancelled

```

db.orders.find({
  category: 'Electronics',
  price: { $gt: 10000 },
  quantity: { $gt: 1 },
  status: { $ne: 'Cancelled' }
})

```

Example 4: Query combining array, embedded field, and comparison operators

```

// BTechCSE students in Delhi who know Python AND have marks between 70 and 95:
db.students.find({
  course: 'BTechCSE',
  'address.city': 'Delhi',
  skills: 'Python',
  marks: { $gte: 70, $lte: 95 }
})

```

8.5 Combine Conditions in Update Operations

```
// Update students who are Active AND know Python AND marks > 80:
db.students.updateMany(
  {
    status: 'Active',
    skills: 'Python',
    marks: { $gt: 80 }
  },
  { $set: { grade: 'A+' }, $push: { tags: 'Winner' } }
)

// Delete orders that are Cancelled OR Pending with price < 100:
db.orders.deleteMany({
  $and: [
    { status: { $in: ['Cancelled','Pending'] } },
    { price: { $lt: 100 } }
  ]
})
```

8.6 Combining Conditions in Aggregation \$match Stage

```
db.students.aggregate([
  {
    $match: {
      $and: [
        { status: 'Active' },
        { course: { $in: ['BTechCSE','BTechDS'] } },
        { $or: [
          { marks: { $gt: 80 } },
          { skills: { $all: ['Python','MongoDB'] } }
        ] }
      ]
    }
  },
  { $group: { _id: '$course', avg: { $avg: '$marks' } } },
  { $sort: { avg: -1 } }
])
```

★ **Exam Tip:** *Implicit AND = most concise (just list fields). Use explicit \$and ONLY when: (a) combining \$or with other conditions, or (b) multiple conditions on the same field. \$or requires array of condition objects.*

UNIT 4 — MASTER QUICK REVISION TABLE

Topic	Key Fact	Syntax / Example
Batch Processing	Performing operations on multiple documents at once rather than individually	<code>bulkWrite()</code> , <code>insertMany()</code> , <code>initializeUnorderedBulkOp()</code>
Ordered Bulk	Stops on FIRST error — strict sequential order	<code>initializeOrderedBulkOp()</code>
Unordered Bulk	Continues on error — faster, parallel-friendly	<code>initializeUnorderedBulkOp()</code>
<code>bulk.execute()</code>	MUST be called to run bulk operations	<pre>var bulk = db.col.initializeUnorderedBulkOp(); bulk.insert(...); bulk.execute();</pre>
Aggregation	Process multiple docs and return computed results	<code>db.col.aggregate([stages])</code>
<code>\$match</code>	Filter documents — place FIRST for performance	<code>{ \$match: { status:'A' } }</code>
<code>\$project</code>	Select specific fields	<code>{ \$project: { name:1, _id:0 } }</code>
<code>\$group</code>	Group docs by a field, apply accumulators	<code>{ \$group: { _id:'\$course', count:{ \$sum:1 } } }</code>
<code>\$sort</code>	Sort documents	<code>{ \$sort: { marks:-1 } }</code> // -1 desc, 1 asc
<code>\$skip</code>	Skip N documents	<code>{ \$skip: 10 }</code>
<code>\$limit</code>	Limit to N documents	<code>{ \$limit: 5 }</code>
<code>\$unwind</code>	One doc per array element	<code>{ \$unwind: '\$skills' }</code>
<code>\$out</code>	Write results to new collection	<code>{ \$out: 'newCollection' }</code>
<code>\$sum</code>	Sum numeric values	<code>totalMarks: { \$sum: '\$marks' } count: { \$sum: 1 }</code>
<code>\$avg</code>	Average of values	<code>avgMarks: { \$avg: '\$marks' }</code>
<code>\$min / \$max</code>	Min or max value in group	<code>minMarks: { \$min: '\$marks' }</code>
<code>\$first / \$last</code>	First or last doc in group	<code>firstStudent: { \$first: '\$name' }</code>
MapReduce	Aggregation for large data — map groups, reduce operates	<code>db.col.mapReduce(mapFn, reduceFn, {out:'result'})</code>
<code>distinct()</code>	Single-purpose: unique values	<code>db.students.distinct('course')</code>
<code>countDocuments()</code>	Single-purpose: count matching docs	<code>db.students.countDocuments({status:'Active'})</code>
<code>createIndex()</code>	Create an index on a field	<code>db.col.createIndex({ marks:1 })</code>
<code>dropIndex()</code>	Drop a specific index	<code>db.col.dropIndex({ marks:1 })</code>
<code>getIndexes()</code>	List all indexes on collection	<code>db.col.getIndexes()</code>
Single Field Index	Index one field — most common	<code>db.col.createIndex({ name:1 })</code>
Compound Index	Index 2+ fields — first field primary	<code>db.col.createIndex({ course:1, marks:-1 })</code>
Multikey Index	Auto-created for array fields	<code>db.col.createIndex({ skills:1 })</code>
Geospatial 2D	Planar geometry	<code>db.places.createIndex({ loc:'2d' })</code>

Geospatial 2DSphere	Spherical geometry	<code>db.places.createIndex({ loc:'2dsphere' })</code>
Text Index	Full-text search on strings	<code>db.col.createIndex({ content:'text' })</code>
Replication	Maintaining multiple copies of data across servers for high availability	<code>rs.initiate()</code> , <code>rs.status()</code> , <code>rs.add()</code>
Primary Node	Receives ALL write operations, maintains oplog	One per replica set at any time
Secondary Node	Replicates from primary via oplog asynchronously, can serve reads	Eligible to become primary on failure
Oplog	Capped collection on primary — records all writes chronologically	Secondaries poll oplog to stay in sync
Failover	Automatic election when primary fails — secondary becomes new primary	<code>rs.stepDown()</code> forces election
Min Replica Set Size	Minimum 3 nodes required	1 Primary + 2 Secondaries (or 2 data + 1 Arbiter)
Dot Notation	Access embedded document fields	<code>db.col.find({ 'address.city': 'Delhi' })</code>
Embedded Exact Match	Entire sub-doc must match — order matters	<code>db.col.find({ address: { city:'Delhi', pin:'110001' } })</code>
Array Element Query	Find if array contains value	<code>db.col.find({ skills: 'Python' })</code>
<code>\$all</code>	Array must contain ALL listed values	<code>{ skills: { \$all: ['Python','MongoDB'] } }</code>
<code>\$size</code>	Array has exact element count	<code>{ skills: { \$size: 3 } }</code>
<code>\$elemMatch</code>	Array element satisfies multiple conditions	<code>{ scores: { \$elemMatch: { subject:'Math', marks:{ \$gt:80 } } } }</code>
<code>\$in</code> on array	Array contains at least one listed value	<code>{ skills: { \$in: ['Python','Java'] } }</code>
<code>\$and</code> implicit	List conditions — all must match	<code>{ course:'CSE', marks:{ \$gt:75 } }</code>
<code>\$and</code> explicit	Required when combining <code>\$or</code> + other	<code>{ \$and: [{city:'Delhi'}, { \$or:[...]}] }</code>
<code>\$or</code>	At least one condition matches	<code>{ \$or: [{marks:{ \$gt:90 }}, {skills:'Python'}] }</code>
<code>\$nor</code>	None of the conditions match	<code>{ \$nor: [{cond1}, {cond2}] }</code>
<code>\$not</code>	Negates a condition	<code>{ marks: { \$not: { \$gt:80 } } }</code>

All the Best for Your End Semester Examination!

UNIT IV — Advanced MongoDB | 23CAF3010 NoSQL & MongoDB | SRM University Delhi-NCR

Batch Processing | Data Aggregation | Indexing | Replica Sets | Embedded Doc Queries | Array Queries | Operators | Combined Conditions